Høgskolen på Vestlandet

ML assignment 2 - Phishing-deteksjon

Bidragsytere

Jørn Kvittem
Torgeir Hellebust

30. oktober 2026

Innhold

1 Beskriv problemet **2**
- 1.1 Omfang . . . 2
- 1.2 Mål . . . 2
- 1.3 Hvorfor maskinlæring . . . 2
- 1.4 Avgrensning . . . 2
- 1.5 Metrikker . . . 2
  - 1.5.1 Maskinlæringsmetrikker . . . 2
  - 1.5.2 Software-metrikker . . . 3
  - 1.5.3 Suksesskriterier . . . 3

2 Data **4**
- 2.1 Datakilde og avgrensning . . . 4
- 2.2 Utforskende analyse og datakvalitet . . . 4
- 2.3 Trenings, validerings og testdata . . . 5
- 2.4 Etikk og begrensninger . . . 5

3 Modellering **6**
- 3.1 Oppsett . . . 6
- 3.2 Baselines . . . 6
- 3.3 Eksperimenter . . . 6
- 3.4 Valg av modell og driftspunkt . . . 7
- 3.5 Feilanalyse . . . 9
- 3.6 Evaluering på testsettet . . . 10
- 3.7 Begrensninger . . . 10

4 Deployment **10**
- 4.1 Løsning . . . 10
- 4.2 Grensesnitt og risikokommunikasjon . . . 11
- 4.3 Testing og responstid . . . 11
- 4.4 Vedlikehold og videre arbeid . . . 11

Bruk av KI **12**

Referanser **12**

1 Beskriv problemet

1.1 Omfang

1.2 Mål

Målet med prosjektet er å utvikle en nettside der brukeren kan lime inn en lenke, for eksempel fra en mistenkelig e-post, og få en vurdering av om lenken sannsynligvis fører til en phishing-side. Resultatet presenteres sammen med modellens estimerte sannsynlighet for phishing. Vurderingen skal være et hjelpemiddel, ikke en garanti for at en lenke er trygg.

1.3 Hvorfor maskinlæring

Svartelister er raske for kjente nettsteder, men krever vedlikehold og kan ikke oppdage nye angrep som ikke er registrert ennå [4]. En maskinlæringsmodell kan lære mønstre i hvordan phishing-adresser er bygget opp. Relevante kjennetegn kan for eksempel være uvanlig lengde, mange underdomener, bruk av sifre, merkenavn på uvanlige steder og IP-adresser som vertsnavn. Hvilke egenskaper modellen faktisk legger vekt på, undersøkes i modelleringsdelen. Modellen kan dermed bidra til å oppdage tidligere ukjente adresser, men kan også feilklassifisere legitime lenker.

1.4 Avgrensning

Datasettet PhreshPhish [8] består av nettsider med tilhørende URL og HTML. I dette prosjektet brukes bare URL-en, og løsningen besøker ikke nettsiden lenken peker til. Dette begrenser ressursbehovet og unngår kontakt med potensielt skadelige nettsider.

Avgrensningen betyr samtidig at modellen ikke analyserer sideinnhold, innloggingsskjemaer eller andre opplysninger i HTML-en. Modellene i den opprinnelige artikkelen [4] bruker derimot både URL og HTML. Den undersøker heller ikke om lenken videresender brukeren til en annen adresse.

Datasettet inneholder nettsider, ikke e-poster. Produktet er derfor en lenkesjekk som blant annet kan brukes på lenker fra mistenkelige e-poster, og ikke et e-postfilter. Prosjektet omfatter ikke analyse av e-posttekst, avsenderinformasjon eller vedlegg.

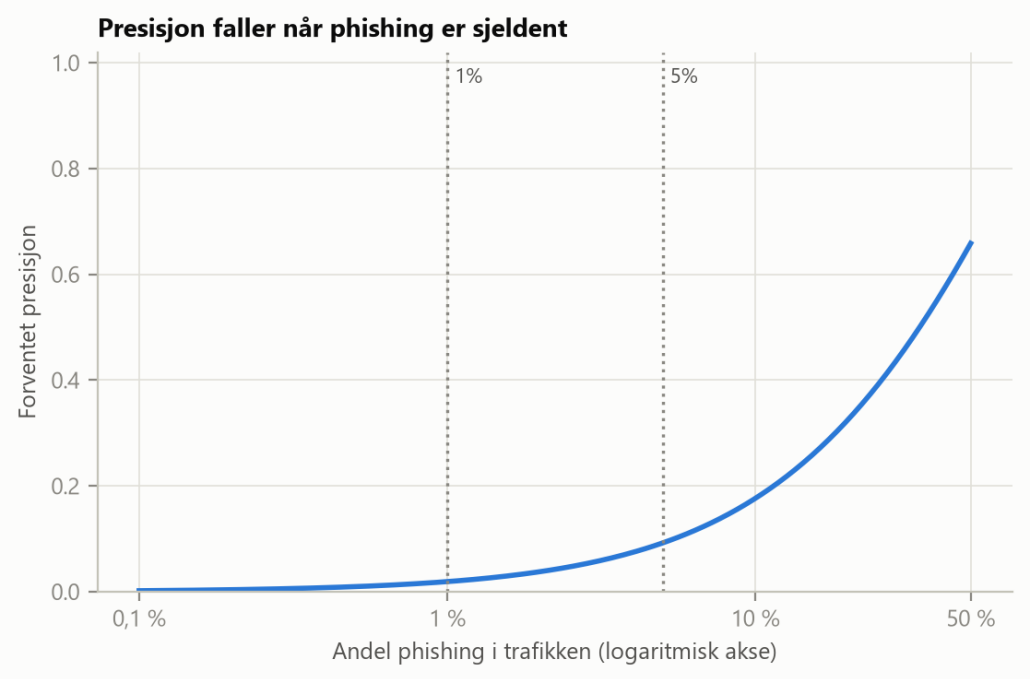
1.5 Metrikker

1.5.1 Maskinlæringsmetrikker

Vi rapporterer presisjon, gjenkalling og F1-skår for phishing-klassen, samt forvekslingsmatrise og average precision (AP). AP oppsummerer forholdet mellom presisjon og gjenkalling på tvers av klassifiseringsterskler.

Treffsikkerhet alene er ikke tilstrekkelig, fordi feiltypene har ulike konsekvenser. En phishing-lenke som ikke oppdages, kan føre til at brukeren oppgir sensitiv informasjon. En legitim lenke som flagges, kan svekke tilliten til verktøyet. Vi vektlegger derfor gjenkalling noe tyngre enn presisjon, men ønsker samtidig å begrense antallet falske alarmer.

Testsettet inneholder omtrent 46 % phishing. I virkelig trafikk kan andelen være langt lavere. Når phishing forekommer sjeldnere, vil presisjonen falle dersom gjenkalling og falsk-positiv-raten er uendret [4]. Vi rapporterer derfor også estimert presisjon ved 1 og 5 % phishing ved å vekte om klassene i testsettet. Beregningen krever ikke nye data, men forutsetter at resultatene innenfor hver klasse er representative også ved den nye klassefordelingen. Figur 1 viser hvordan forventet presisjon faller når andelen phishing synker.



Figur 1: Forventet presisjon når andelen phishing i trafikken er lavere enn i datasettet. Regnet ut fra gjenkalling og falsk-positiv-rate på valideringssettet ved terskelen for 90 % gjenkalling. De stiplede linjene markerer 1 og 5 %.

1.5.2 Software-metrikker

Vi måler responstid for URL fra brukeren sender inn lenken til resultatet vises. Målet er at minst 95 % av forespørslene besvares innen ett sekund når tjenesten er startet og behandler en forespørsel om gangen. Oppstartstid rapporteres separat.

Vi måler også modellens filstørrelse og kontrollerer at hele applikasjonen kan kjøres innenfor ressursgrensene til en gratis Hugging Face Space. Siden løsningen ikke henter nettsiden, unngår den ventetid knyttet til dette. Responstiden omfatter likevel nettverkskommunikasjon, uttrekk av URL-egenskaper og prediksjon.

1.5.3 Suksesskriterier

Målet er å hjelpe brukere med å unngå phishing-lenker. Vi undersøker ikke faktisk klikkatferd, og modellens resultater brukes derfor som et indirekte mål på nytte.

Det tekniske målet er minst 90 % presisjon ved 90 % gjenkalling for phishing-klassen. Dette tilsvarer metrikken presisjon ved recall 0,9 i PhreshPhish-artikkelen [4]. Sammenligningen må ta hensyn til at prosjektet bare bruker URL-er, mens alle modellene i artikkelen bruker både URL og HTML.

Klassifiseringsterskelen velges på valideringssettet med sikte på 90 % gjenkalling. Deretter rapporteres både presisjon og faktisk oppnådd gjenkalling på testsettet. Vi rapporterer også presisjon ved 90 % gjenkalling fra testsettets presisjon-gjenkalling-kurve for sammenligning med artikkelen.

Dersom valideringsresultatene viser at målet er urealistisk med URL alene, kan det revideres før den endelige testevalueringen. Eventuelle endringer begrunnes i modelleringsdelen, og både det opprinnelige og det reviderte målet oppgis.

2 Data

2.1 Datakilde og avgrensning

Vi bruker datasettet PhreshPhish [8]. Datasettet inneholder URL-er og HTML fra legitime nettsider og phishing-sider. Det består av nettsider, ikke e-poster. I prosjektet bruker vi bare URL-teksten. Modellen undersøker verken innholdet på nettsiden eller e-posten lenken eventuelt kommer fra.

Vi bruker versjon 1.0.1 av datasettet. Den har en forhåndsdefinert trenings- og testdeling og til sammen 666 315 rader [8]. Antallene gjelder denne versjonen, og artikkelen [4] oppgir en lavere total (613 785 rader etter rensing). Fordelingen er vist i tabell 1.

Del	Legitime	Phishing	Totalt
Trening	276 729	221 526	498 255
Test	91 260	76 800	168 060
Totalt	**367 989**	**298 326**	**666 315**

Tabell 1: Fordeling av klasser i datasettets opprinnelige deling.

Vi henter kolonnene `url`, `label`, `date` og `target`. Dette reduserte datamengden til omtrent 20 MB, uten å laste ned HTML-kolonnen. Datasettkortet [8] oppgir at hele datasettet er på omtrent 145 GB for trening og 48 GB for test. URL og klasseetikett brukes til modellering, mens dato og målmerke brukes til deling og analyse. En kontroll fant ingen identiske URL-er i datasettet og ingen identiske URL-er på tvers av trening og test. Nesten-duplikater er ikke undersøkt.

2.2 Utforskende analyse og datakvalitet

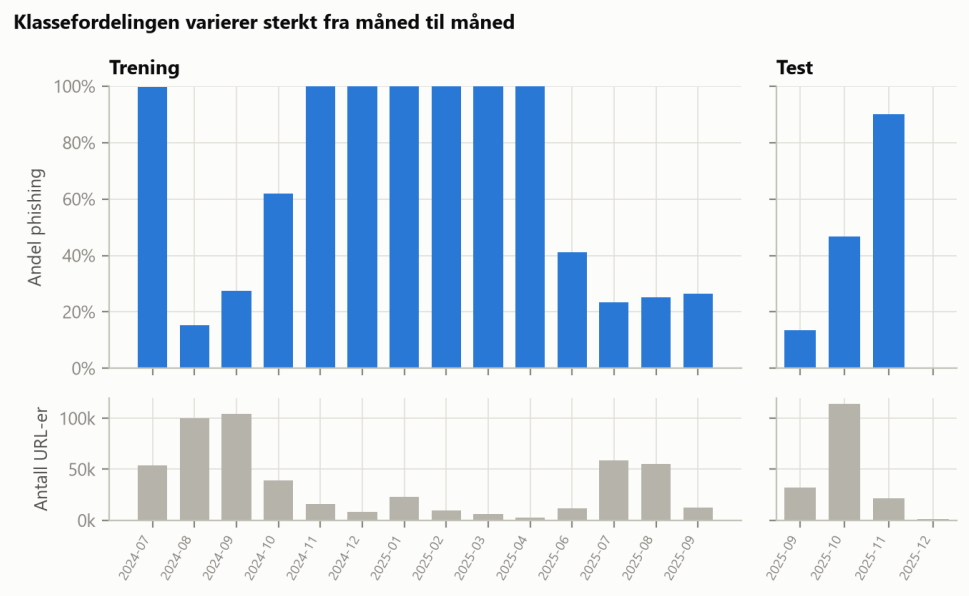
Klassefordelingen varierer betydelig over tid. I treningsdelen varierer andelen phishing per måned fra omtrent 15 til 100 %, og legitime URL-er finnes bare i enkelte måneder. Figur 2 viser denne variasjonen. En tidsbasert deling kan derfor innebære endringer både i klassefordeling og i hvilke datakilder og phishing-kampanjer modellen møter.

Analysen av treningsdelen viste også forskjeller i skrivemåte mellom klassene (tabell 2).

Egenskap	Legitime	Phishing
Inneholder `www.`	58,6 %	4,0 %
Slutter på `/`	33,9 %	43,7 %
Har ingen sti	7,0 %	44,6 %
Har spørringsdel (`?`)	0,0 %	7,4 %

Tabell 2: Skrivemåteegenskaper i den opprinnelige treningsdelen (andel av URL-ene i hver klasse).

Disse forskjellene kan gi modellen nyttige signaler, men kan også være artefakter fra datainnsamlingen. For eksempel kan modellen lære at `www.` kjennetegner legitime URL-er, uten at dette er et pålitelig sikkerhetssignal i virkelig bruk. Blant phishing-URL-ene varierer andelen med avsluttende skråstrek mellom 29 og 71 % per måned. Dette motiverte forsøk med normalisering av URL-ene, beskrevet i modelleringskapitlet.



Figur 2: Andel phishing (øverst) og antall URL-er (nederst) per måned. Figuren viser hele datasettet, også testmånedene, og er bare beskrivende. Legitime URL-er mangler i flere måneder.

Et særlig tydelig artefakt gjelder spørringsdelen. Artikkelen [4] oppgir at personvernarbeidet ble rettet mot de legitime sidene, og at søkeparametere ble fjernet fra deres URL-er. Nesten ingen legitime adresser i treningsdelen har derfor en spørringsdel (0,01 %), mot 7,4 % av phishing-adressene. Modellen kan lære at en spørringsdel peker mot phishing, noe som ikke stemmer for vanlige lenker, for eksempel lenker i nyhetsbrev med sporingsparametere.

2.3 Trenings, validerings og testdata

Vi satte av de nyeste 20 % av den opprinnelige treningsdelen til validering, sortert etter dato. Det ga 398 604 rader til modelltrening og 99 651 til validering. Phishing utgjør omtrent 28,3 % av valideringsdelen.

Formålet med denne delingen er å undersøke hvordan modellen håndterer senere observasjoner fremfor et tilfeldig utvalg fra samme periode. Valideringsdelen brukes til modellvalg og valg av beslutningsterskel. Testdelen holdes utenfor disse valgene og brukes til avsluttende evaluering. Delingen er likevel ikke fullstendig adskilt. Artikkelen beskriver testsettet som strengt senere enn treningssettet [4], men i versjon 1.0.1 av datasettet inneholder både den opprinnelige treningsdelen og testdelen observasjoner fra september 2025.

2.4 Etikk og begrensninger

Ifølge artikkelen [4] er personvernarbeidet rettet mot de legitime sidene: søkeparametere er fjernet fra URL-er, og adresser som kan inneholde navnet på en privatperson er filtrert bort. De legitime adressene bygger på anonymisert nettleserdata, men annen personinformasjon, som e-postadresser og telefonnumre i HTML-en, er ikke forsøkt fjernet. Datasettkortet [8] oppgir lisensen CC BY 4.0 og en bruksbegrensning til anti-phishing-forskning.

Forfatterne [4] beskriver en kombinasjon av automatisk filtrering og delvis manuell gjennomgang, der en annotatør vurderer ett representativt eksempel per gruppe, og de påstår ikke at datasettet er feilfritt. Vi kan derfor ikke anta at alle merkinger er korrekte. Resultatene må også tolkes i lys av tidsvariasjonen og innsamlingsmetoden: gode resultater på dette datasettet dokumenterer ikke alene tilsvarende ytelse på lenker i vanlig e-posttrafikk.

3 Modellering

3.1 Oppsett

Modellen er en scikit-learn-`Pipeline` [7, 3]. Adressen normaliseres først: små bokstaver, uten skjema, uten `www.` først i adressen og uten avsluttende skråstrek. Deretter beregnes tre typer egenskaper som settes sammen: 18 håndlagde tall (blant annet lengder, antall punktum, bindestreker og sifre, IP-adresse som vert, entropi i vertsnavnet og mistenkelige ord), TF-IDF over tegn-n-grammer på 3–5 tegn (maks 300 000) og TF-IDF over hele ord (maks 200 000). Til slutt gir en logistisk regresjon en modellscore mellom 0 og 1.

Valideringsdelen fra kapittel 2.3 brukes til alle valg. Alle kjøringer bruker samme data og samme seed (42), og hver variant er kjørt en gang. Vi har ikke målt tilfeldig variasjon, så forskjeller på under omtrent 0,01 AP regnes som usikre.

3.2 Baselines

Majority-baselinen, som alltid svarer legitim, får AP 0,283. Det er andelen phishing i valideringsdelen, og viser at treffsikkerhet alene ikke sier noe. Regelbaselinen gir poeng for IP-adresse som vert, @ i adressen, URL lengre enn 75 tegn, flere enn to underdomener og mistenkelige ord. Poengene er grove heltall, så terskelen som gir 90 % gjenkalling havner på null og regelen flagger alle adresser (presisjon 0,283, AP 0,354). Reglene er for grove til å skille klassene ved høy gjenkalling. Første modell, med logistisk regresjon ($C = 1$) og uten normalisering og ordegenskaper, får AP 0,887 og presisjon 0,480 ved 90 % gjenkalling.

3.3 Eksperimenter

Tabell 3 viser hva vi endret og hvordan valideringsresultatene ble påvirket. Vi endret en ting om gangen.

Endring	AP	Presisjon ved 90 % gjenkalling
Første modell	0,887	0,480
Normalisert URL	0,873	0,395
Normalisert URL og ordegenskaper	0,877	0,400
$C = 0{,}3$	0,866	0,374
$C = 3$	0,881	0,415
$C = 10$ (valgt)	**0,882**	**0,433**
$C = 30$	0,879	0,411
$C = 100$	0,879	0,406
LinearSVC med kalibrering	0,883	0,425
Tegn-n-grammer 4–6	0,875	0,392
Gradient boosting, bare de 18 tallene	0,814	0,392

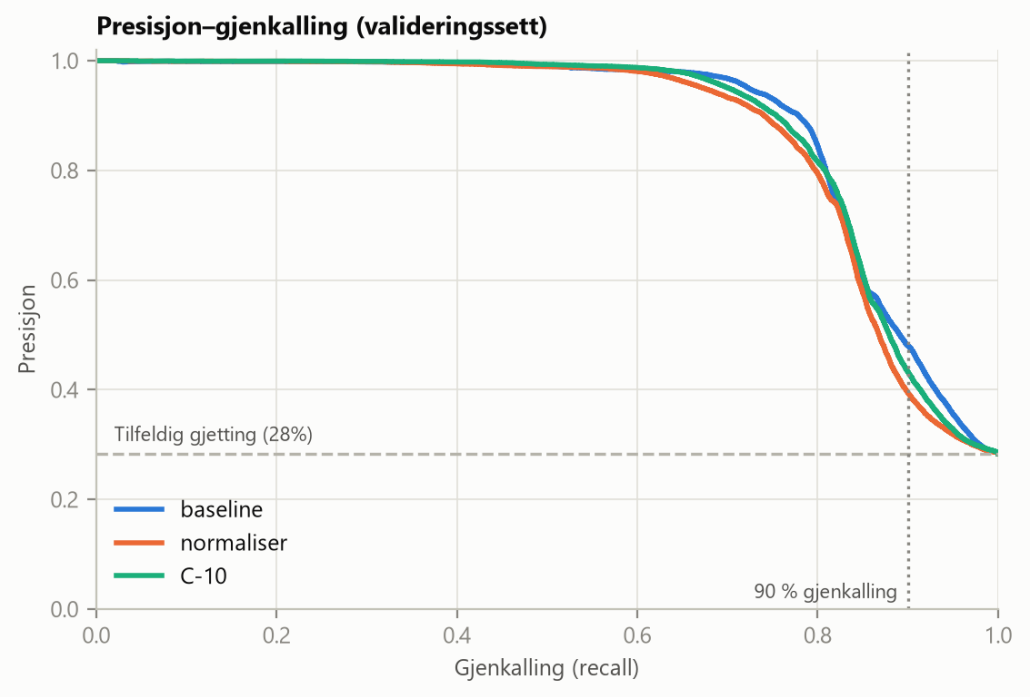
Tabell 3: Resultater på valideringssettet. Linjene med C, LinearSVC og n-grammer er varianter av «Normalisert URL og ordegenskaper». Gradient boosting er en variant av «Normalisert URL».

Normalisering. Skrivemåteforskjellene i kapittel 2.2 tyder på at modellen kan lære hvordan dataene ble samlet inn, ikke hva phishing er. Første modell endret vedtak for 9,4 % av 1 000 valideringsadresser bare ved at de ble skrevet som `https://www..../` i stedet for uten skjema og `www.`. Etter normalisering er dette 0 %. Prisen var lavere valideringsscore: AP falt fra 0,887 til 0,873, og presisjonen ved 90 % gjenkalling fra 0,480 til 0,395. Vi beholdt normaliseringen fordi

brukere limer inn lenker i mange former, og fordi andelen phishing-adresser med avsluttende skråstrek varierer mellom 29 og 71 % fra måned til måned. Slike mønstre er ikke pålitelige over tid.

Ordegenskaper og regularisering. Hele ord ga en liten forbedring (AP +0,004). Av regulariseringsstyrkene fra C = 0,3 til 100 ga C = 10 best resultat. Sterk regularisering var dårligst. Forskjellene er små og ligger i området vi regner som usikkert.

Andre modeller. LinearSVC er en av basismodellene i PhreshPhish-artikkelen [4]. Vi brukte den med kalibrering for å få sannsynligheter, og den ga samme resultat som logistisk regresjon, men trente lenger (242 mot 135 sekunder). Gradient boosting på bare de 18 tallene ga lavere AP, men nesten samme presisjon ved 90 % gjenkalling, og trente på 12 sekunder. Lengre tegn-n-grammer ga ingen forbedring. Det viktigste funnet er at modellvalg og tuning betyr lite: alle lineære varianter ligger mellom 0,87 og 0,88 AP.



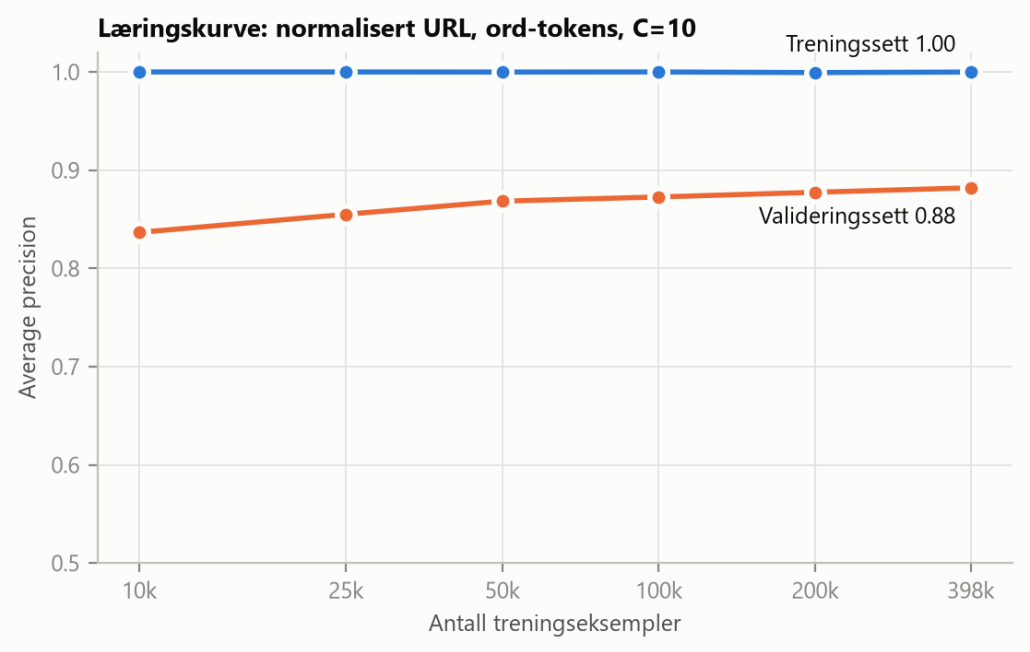
Figur 3: Presisjon mot gjenkalling på valideringssettet for første modell (baseline), modellen med normalisert URL og valgt modell (C-10). Den stiplede linjen viser andelen phishing (0,28), og den prikkede linjen markerer 90 % gjenkalling.

Læringskurven i figur 4 viser at modellen husker treningsdataene: AP på treningsdata er omtrent 1,00 uansett datamengde, mens AP på validering stiger sakte fra 0,837 med 10 000 eksempler til 0,882 med alle 398 604. Mer data fra samme periode vil trolig hjelpe litt. Gapet skyldes også at valideringsperioden inneholder kampanjer som treningsdelen ikke har (se avsnitt 3.5).

3.4 Valg av modell og driftspunkt

Vi velger logistisk regresjon med normalisert URL, ordegenskaper og C = 10. Den er enkel å forklare, rask og ikke dårligere enn LinearSVC. Beslutningsterskelen er satt på valideringssettet slik at gjenkallingen blir 90 %, som beskrevet i kapittel 1. Terskelen ble 0,0007.

Kravet i kapittel 1.5.3 om minst 90 % presisjon ved 90 % gjenkalling nås ikke på valideringssettet. Presisjonen er 0,43 for valgt modell. Tabell 4 viser at den faller bratt mellom 80 og 90 % gjenkalling (0,82 mot 0,43). Vi reviderer derfor det tekniske målet til minst 80 % presisjon ved 80 % gjenkalling, som valgt modell oppfyller (0,82). Det opprinnelige målet er ikke oppnådd.



Figur 4: Læringskurve for valgt modell. AP på treningsdata er målt på et utvalg av treningsdataene selv.

Gjenkalling	0,5	0,7	0,8	0,9	0,95
Første modell	0,99	0,97	0,85	0,48	0,36
Valgt modell	0,99	0,95	0,82	0,43	0,33

Tabell 4: Presisjon ved ulike gjenkallinger på valideringssettet.

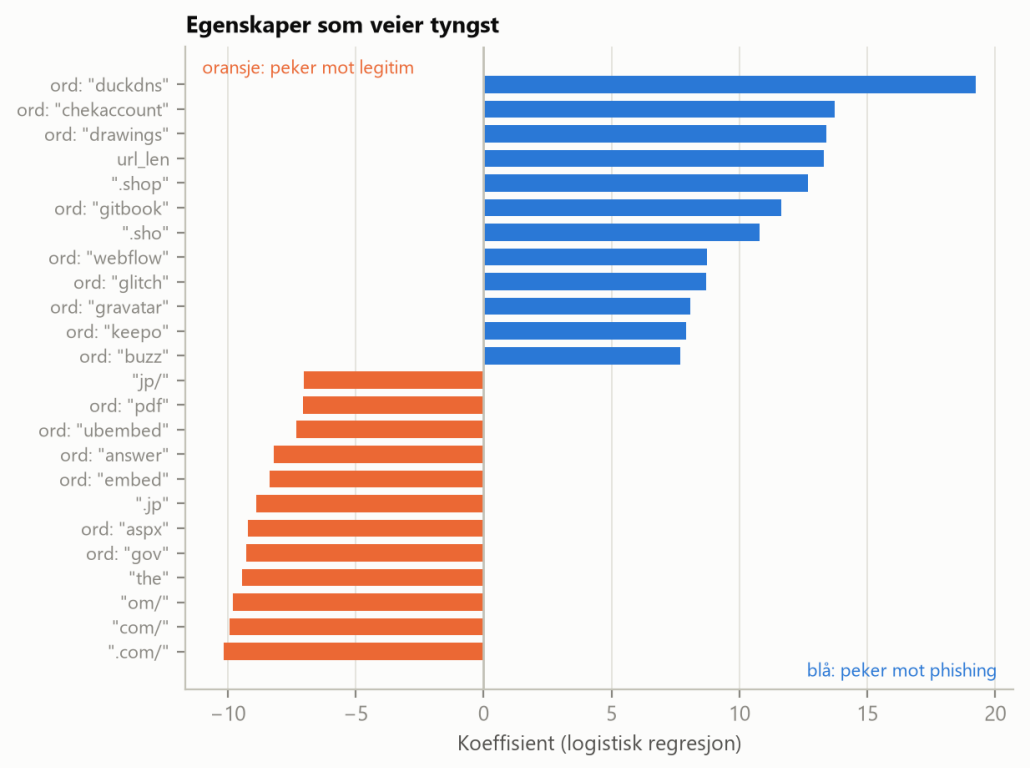
Med 1 % phishing i trafikken er forventet presisjon ved terskelen for 90 % gjenkalling om lag 2 %, og ved 5 % om lag 9 % (figur 1).

3.5 Feilanalyse

Ved terskelen for 90 % gjenkalling bommer modellen på 2 825 av 28 250 phishing-adresser i valideringssettet. Vi grupperte dem etter merkenavnet i kolonnen `target`. 2 394 tilhører `robinhood` (av 5 067, hvorav 53 % fanges) og 386 tilhører `qmp` (av 448, hvorav 14 % fanges). Til sammen er det 98 % av alle bom. Ingen av de to merkene finnes i treningsdelen, og 338 merkenavn i valideringsperioden mangler helt der. Adressene ligner vanlige sider, for eksempel forumtråder, hjelpesentre og produktsider, og 15 % av alle phishing-adressene i valideringssettet får en modellscore under 0,01.

For å forstå hvor mye disse to kampanjene betyr, regnet vi ut metrikkene uten dem, som er 5,5 % av valideringsradene. AP er da 0,973 og presisjonen ved 90 % gjenkalling 0,921. Dette er en følsomhetsanalyse og ikke et resultat for modellens ytelse. Å fjerne vanskelige eksempler etter å ha sett resultatene er ikke en gyldig forbedring. Analysen viser likevel at presisjon ved 90 % gjenkalling i stor grad måler hvor godt modellen takler de vanskeligste ukjente kampanjene. For å fange dem måtte terskelen settes så lavt at 47 % av de legitime adressene også flagges. Vi har ikke undersøkt om etikettene for disse adressene er riktige.

De egenskapene som veier tyngst (figur 5) peker mot gratis tjenester for nettsider (`duckdns`, `webflow`, `glitch`, `gitbook`) og mot lange adresser. `.com/`, `gov`, `pdf` og `.jp` peker mot legitim. At `.jp` veier mot legitim kan tyde på at hvor de legitime adressene kommer fra, påvirker modellen.



Figur 5: Egenskapene med størst koeffisient i valgt modell. Blå peker mot phishing og oransje mot legitim. Koeffisientene for tall og tekstgrener er ikke direkte sammenlignbare.

3.6 Evaluering på testsettet

Testsettet ble brukt en gang for valgt modell, med terskelen fra valideringssettet. Vi kjørte i tillegg første modell for å sjekke om normaliseringen hjelper på senere data. Dette påvirket ikke valget av modell.

Måling på testsettet	Valgt modell	Første modell
Average precision	0,978	0,981
Gjenkalling ved valideringsterskelen	0,981	0,982
Presisjon ved valideringsterskelen	0,627	0,669
Falsk-positiv-rate ved valideringsterskelen	0,491	0,408
Presisjon ved 90 % gjenkalling (egen terskel)	0,960	0,971
Presisjon ved 1 % phishing i trafikken	0,020	0,024

Tabell 5: Resultater på testsettet.

Hypotesen om at normaliseringen ville hjelpe på senere data ble ikke bekreftet. Første modell er like god eller litt bedre (AP 0,981 mot 0,978, innenfor det vi regner som usikkert). Normaliseringen ga robusthet, men ikke bedre treffsikkerhet. AP er mye høyere på testsettet (0,98) enn på valideringssettet (0,88). Det skyldes trolig at testperioden er enklere, ikke at modellen er bedre. Terskelen fra validering overføres dårlig: den fanger 98 % av phishing-adressene, men flagger også 49 % av de legitime. Terskelen bør derfor velges ut fra hvilken presisjon eller falsk-positiv-rate vi kan akseptere, og følges opp i drift. Presisjonen ved 1 % phishing er bare om lag 2 %, uansett hvor høy AP er.

3.7 Begrensninger

Resultatene bygger på en seed og har ingen feilmarginer. Modellen ser bare på teksten i adressen og fanger ikke phishing-sider med vanlige adresser. Få kampanjer avgjør tallene, og valideringsperioden og testperioden har ulike kampanjer. Vi har ikke undersøkt om modellscoren er kalibrert, så den kan ikke leses som en sannsynlighet. Nesten-duplikater mellom trening og test er heller ikke undersøkt.

Modellen er også sårbar for adresser med spørringsdel. I en enkel kontroll etter modelleringen fikk vanlige legitime adresser høy modellscore når vi la til sporingsparametere: `hvl.no/studier` økte fra 0,03 til 0,999 med `utm_`-parametere og `fbclid`, og `nrk.no/nyheter` fra 0,01 til 0,94 med `utm_`-parametere. Årsaken er at de legitime adressene i treningsdataene mangler spørringsdel (kapittel 2.2). Resultatene i dette kapitlet sier derfor lite om slike adresser, og de er vanlige i virkelig e-posttrafikk.

4 Deployment

4.1 Løsning

Modellen er en scikit-learn-pipeline lagret med `joblib` (17 MB). Den brukes i en Gradio-app [1] på Hugging Face Spaces [5], og appen laster modellen en gang ved oppstart. På en gratis Hugging Face-konto kan man bare opprette Gradio-Spaces som kjører på ZeroGPU-maskinvare, og andre Gradio-Spaces krever betalt abonnement [5, 6]. Appen er satt opp på ZeroGPU, men ber aldri om GPU.

Normaliseringen ligger inne i pipelinen, så appen bruker nøyaktig de samme stegene som under trening. `src/features.py` er kopiert uendret fra treningsmappen, og scikit-learn-versjonen er

låst til samme versjon i begge mapper, fordi `joblib`-filer kan slutte å virke mellom versjoner [9]. Den lagrede pipelinen refererer til modulen `src.features`, så appen trenger en `src`-mappe med samme navn.

4.2 Grensesnitt og risikokommunikasjon

Brukeren limer inn en nettadresse og får ett resultatkort med overskrift, forklaring, anbefalt neste steg og selve adressen vist som ren tekst. Adressen åpnes aldri. Det finnes tre nivåer som bygger på modellscoren: «Tegn på phishing» fra 0,80, «Mulige tegn på phishing» fra 0,30 og «Ingen tydelige tegn funnet» under det. Grensene er foreløpige, fordi scoren ikke er kalibrert og driftspunktet ikke er endelig. Resultatet kalles modellscore og ikke sannsynlighet, og teksten sier at et lavt utslag ikke er en garanti for at nettstedet er trygt.

Under resultatet kan brukeren åpne en forklaring med de egenskapene som veide mest (vekt ganget med egenskapens verdi i den logistiske regresjonen), og en seksjon med modellscore, grenser og normalisert tekst. Forklaringene beskriver bare mønstre i adressen. Appen kontrollerer ikke eier, alder eller omdømme, og det står i grensesnittet. Eksemplene er delt i vanlige adresser, konstruerte phishing-adresser og to eksempler på modellfeil, slik at brukeren ser både styrker og svakheter. Appen lagrer ikke adressene, skriver dem ikke til loggen og bruker ikke analyseverktøy, fordi adressene kan inneholde personopplysninger i spørringsdelen.

4.3 Testing og responstid

Under første kjøring på hele datasettet krasjet treningen fordi `urlparse` kaster en feil for enkelte ugyldige adresser. Det ville også ha krasjet appen på rare innspill, og feilen ble ikke funnet på et mindre utvalg. Den er rettet ved at ugyldige adresser behandles som tomme. Vi har 13 automatiske kontroller for tom og ugyldig innsending, lavt og høyt utslag, lange adresser, adresser uten skjema, visning av adressen som ren tekst, feil i modellkallet med ny prøving og at gammelt resultat fjernes når adressen endres. Vi har også sett grensesnittet i nettleser på skrivebordsbredde og i 320 piksler bredde, i lyst og mørkt tema. Vi har ikke testet med skjermleser. Appen har samme svakhet som modellen for adresser med sporingsparametere (avsnitt 3.7).

Selve vurderingen i appen, inkludert forklaringen, tok 15–25 ms lokalt, og modellen alene omtrent 1–3 ms. Målt mot Space-en via API fra vår maskin var medianen 1,30 sekunder og 95-persentilen 1,37 sekunder for 40 kall. Et kall til en funksjon som ikke bruker modellen tok like lang tid (median 1,30 sekunder). Tiden domineres dermed av nettverk og plattform og ikke av modellen. Målet i kapittel 1.5.2 om at 95 % av forespørslene besvares innen ett sekund er derfor ikke dokumentert oppfylt, og vi har ikke målt fra nettleseren. Modellfilen på 17 MB lastes uten problemer på Space-en, som kjører på ZeroGPU-maskinvaren uten kostnad.

4.4 Vedlikehold og videre arbeid

Phishing-kampanjer endrer seg raskt. Valideringsanalysen viste at nye kampanjer med vanlige adresser slipper gjennom, og testresultatet viste at en terskel satt på en periode ikke passer en annen. Modellen bør derfor trenes på nyere data med jevne mellomrom, og terskelen bør settes på nytt hver gang. Andelen advarsler kan følges opp uten å lagre selve adressene.

Mulige forbedringer er å ignorere spørringsdelen i adressen, både i trening og i appen, domenesignaler fra en offentlig suffiksliste og et flagg for kjente gratis tjenester for nettsider, vekting av store kampanjer i treningen, kalibrering av scoren, flere seeds for feilmarginer og HTML-egenskaper slik artikkelen bruker [4]. HTML-egenskaper ligger utenfor vår avgrensning.

Bruk av KI

Våre valg

De overordnede valgene har vi tatt selv. Vi valgte problemet, datasettet, avgrensningen til URL-er, Gradio på Hugging Face Spaces og metrikkene og målene i kapittel 1. Vi bestemte også hvordan vi delte datasettet, hvilke modeller og egenskaper vi skulle prøve, og hvordan vi skulle evaluere dem.

Hva verktøyet har gjort

Vi har brukt Claude Code [2] med modellen Claude Sonnet 5.5 som hjelpemiddel under vår styring. På oppdrag fra oss har verktøyet gjort følgende:

- referansekode for databehandling, trening, evaluering og figurer, og kjøring av eksperimentene som er beskrevet i kapittel 3
- bygging av nettsiden i kapittel 4 og testene til den

Verktøyets forslag er ikke tatt for gitt. Flere forslag ble avvist eller endret, og henvisningene til kildene er kontrollert mot det kildene faktisk sier.

Referanser

[1] Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan, and James Zou. Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv preprint arXiv:1906.02569, 2019.

[2] Anthropic. Claude Code. Command-line programming tool, model Claude Sonnet 5.5, 2026. Lest 2026-10-30.

[3] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake Vanderplas, Arnaud Joly, Brian Holt, and Gael Varoquaux. API design for machine learning software: Experiences from the scikit-learn project. arXiv preprint arXiv:1309.0238, 2013.

[4] Thomas Dalton, Hemanth Gowda, Girish Rao, Sachin Pargi, Alireza Hadj Khodabakhshi, Joseph Rombs, Stephan Jou, and Manish Marwah. PhreshPhish: A real-world, high-quality, large-scale phishing website dataset and benchmark. *arXiv preprint arXiv:2507.10854*, 2025. Versjon 2.

[5] Spaces overview. Hugging Face Hub dokumentasjon. Lest 2026-10-30.

[6] Spaces ZeroGPU: Dynamic GPU allocation for Spaces. Hugging Face Hub dokumentasjon. Lest 2026-10-30.

[7] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[8] PhreshPhish dataset card. Hugging Face Datasets. Versjon 1.0.1. Lest 2026-10-26.

[9] Model persistence. scikit-learn brukerveiledning. Versjon 1.9.1. Lest 2026-10-30.